

PDV CoreTech

Documentação Técnica do Sistema

Versão 1.0.0 | Desenvolvido por Core Tech | 2026

Este documento descreve a arquitetura, estrutura de arquivos, módulos e funcionamento interno do sistema PDV CoreTech, desenvolvido em Python com interface desktop PyQt6.

1. Visão Geral da Arquitetura

O PDV CoreTech é um sistema de Ponto de Venda desktop desenvolvido em Python, utilizando PyQt6 para a interface gráfica e SQLite como banco de dados local. A arquitetura segue o padrão de separação de responsabilidades, dividindo o sistema em três camadas principais: core (lógica de negócio), desktop (interface gráfica) e config (configurações).

1.1 Stack Tecnológica

Tecnologia	Versão	Uso
Python	3.14+	Linguagem principal
PyQt6	6.11.0	Interface gráfica desktop
SQLite	Built-in	Banco de dados local
PyInstaller	6.20.0	Geração do executável .exe
Pandas	Latest	Manipulação de dados nos relatórios
Hashlib	Built-in	Criptografia de senhas (SHA-256)

2. Estrutura de Pastas

O projeto segue uma estrutura organizada e modular, separando claramente a lógica de negócio da interface gráfica:

```
pdv-coretech/
├── core/ # Lógica de negócio
│   ├── __init__.py
│   ├── database.py # Conexão e criação das tabelas
│   ├── produtos.py # CRUD de produtos
│   ├── clientes.py # CRUD de clientes
│   ├── estoque.py # Controle de estoque
│   ├── vendas.py # Registro de vendas
│   ├── relatorios.py # Consultas e relatórios
│   ├── usuarios.py # Autenticação e usuários
│   └── caixa.py # Abertura/fechamento de caixa
├── desktop/ # Interface gráfica PyQt6
│   ├── __init__.py
│   ├── main.py # Ponto de entrada do sistema
│   └── telas/
│       ├── tela_login.py
│       ├── tela_caixa.py
│       ├── tela_caixa_operacao.py
│       ├── tela_produtos.py
│       ├── tela_clientes.py
│       └── tela_relatorios.py
├── componentes/
├── config/
│   ├── config.json # Configurações da loja e tema
│   ├── temas/
│   └── assets/ # Logos e ícones
├── comprovante/
│   ├── comprovante.py # Geração do cupom
│   └── impressao/
│       ├── impressora.py # Integração com impressora térmica
│       └── web/
│           ├── app.py # Versão web (Streamlit)
│           ├── requirements.txt
│           └── README.md
```

3. Módulos do Core

3.1 database.py

Responsável pela conexão com o banco de dados SQLite e pela criação de todas as tabelas do sistema. A função `conectar()` retorna uma conexão ativa e é inteligente o suficiente para funcionar tanto no modo

desenvolvimento quanto no executável .exe gerado pelo PyInstaller.

Tabelas criadas:

Tabela	Descrição
produtos	Cadastro de produtos com código, nome, preço e estoque
clientes	Cadastro de clientes com CPF, telefone e endereço
vendas	Registro de cada venda com total, desconto e forma de pagamento
itens_venda	Itens individuais de cada venda (produto, qtd, preço)
usuarios	Usuários do sistema com login e senha criptografada
caixa	Registro de abertura e fechamento do caixa
movimentacao_caixa	Sangrias, suprimentos e movimentações do caixa
movimentacao_estoque	Histórico de entradas e saídas do estoque

3.2 produtos.py

Gerencia o cadastro completo de produtos. Principais funções:

3.3 vendas.py

Módulo central do sistema. A função **registrar_venda()** utiliza uma única conexão com o banco de dados para registrar a venda, os itens e atualizar o estoque em uma única transação, garantindo consistência e alta performance.

Fluxo de uma venda:

1. Recebe a lista de itens, forma de pagamento e desconto
2. Calcula o total com desconto
3. Insere o registro na tabela **vendas**
4. Para cada item: insere em **itens_venda** e baixa o estoque em **produtos**
5. Registra a movimentação em **movimentacao_estoque**
6. Realiza o commit de tudo em uma única transação
7. Em caso de erro, realiza rollback automático

3.4 usuarios.py

Gerencia a autenticação e o controle de acesso. As senhas são armazenadas com criptografia **SHA-256** — nunca em texto puro. O usuário administrador padrão é criado automaticamente na primeira execução do sistema.

Credenciais padrão:

- Login: **admin**
- Senha: **admin123**

Importante: altere a senha padrão antes de entregar o sistema ao cliente!

3.5 caixa.py

Controla o ciclo de vida do caixa: abertura, fechamento, sangrias e suprimentos. Todas as movimentações são registradas na tabela **movimentacao_caixa** com data, hora e tipo.

4. Interface Desktop (PyQt6)

A interface é construída com PyQt6 e organizada em telas independentes, gerenciadas pelo **QStackedWidget** no `main.py`. Cada tela é um widget independente que recebe o dicionário de configurações do cliente.

4.1 `main.py` — Ponto de entrada

Responsável por inicializar o banco de dados, criar o usuário admin, exibir a tela de login e, após autenticação bem-sucedida, carregar a janela principal com o menu lateral e as telas empilhadas.

4.2 Tela de Login (`tela_login.py`)

Exibe o formulário de login com o nome da loja carregado do `config.json`. Emite o sinal **login_sucesso** com os dados do usuário ao autenticar corretamente, permitindo que o `main.py` abra a janela principal com o contexto do usuário logado.

4.3 Tela do Caixa (`tela_caixa.py`)

Principais funcionalidades:

- Busca de produto por código ou nome com autocomplete
- Campo de quantidade com foco automático após busca
- Enter no campo quantidade adiciona o item automaticamente
- Edição de quantidade diretamente na tabela (duplo clique)
- Cálculo automático de total e desconto em tempo real
- Seleção de forma de pagamento
- Finalização rápida com geração de comprovante
- Retorno automático ao campo de busca após finalizar

4.4 Personalização por cliente

Toda a personalização visual e de dados da loja é feita através do arquivo **config/config.json**. Para cada novo cliente, basta editar esse arquivo com o nome, endereço, CNPJ, cores e logo da loja, e gerar um novo `.exe`.

Estrutura do `config.json`:

```
{ "loja": { "nome": "Nome da Loja", "slogan": "Slogan da loja", "cnpj":  
"00.000.000/0000-00", "telefone": "(84) 99999-9999", "endereco": "Rua, número,  
bairro" }, "tema": { "cor_primaria": "#6c63ff", "cor_secundaria": "#00d4aa",  
"cor_fundo": "#0a0a0f", "cor_texto": "#e8e8f0" } }
```

5. Geração do Executável (.exe)

O sistema usa o **PyInstaller** para gerar um executável Windows independente. O comando abaixo empacota todos os arquivos necessários em um único `.exe`:

```
python -m PyInstaller --onefile --windowed --name "PDV-CoreTech" --add-data  
"config/config.json;config" --add-data "assets;assets" --add-data "core;core"  
--add-data "desktop;desktop" --add-data "comprovante;comprovante" desktop/main.py
```

Flags utilizadas:

- **--onefile**: empacota tudo em um único arquivo .exe
- **--windowed**: esconde o terminal ao executar
- **--name**: define o nome do executável
- **--add-data**: inclui arquivos e pastas no pacote

O executável gerado fica na pasta **dist/** e pode ser distribuído diretamente ao cliente sem necessidade de instalar Python ou qualquer dependência.